IDUG

2026

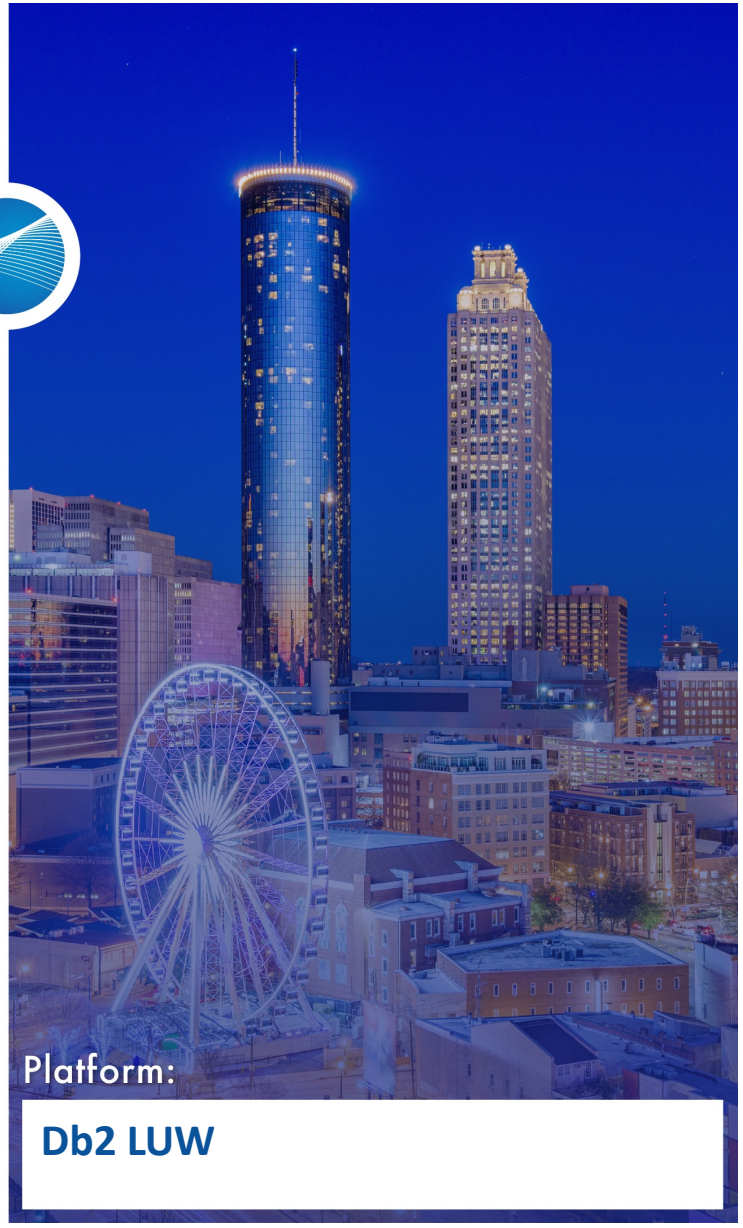
Atlanta | October 26 - 29

NA Db2 TECH CONFERENCE

Bridging Structured and Unstructured Data: Hybrid AI Queries with Db2

Wilfried Hoge & Andreas Weininger, *IBM*

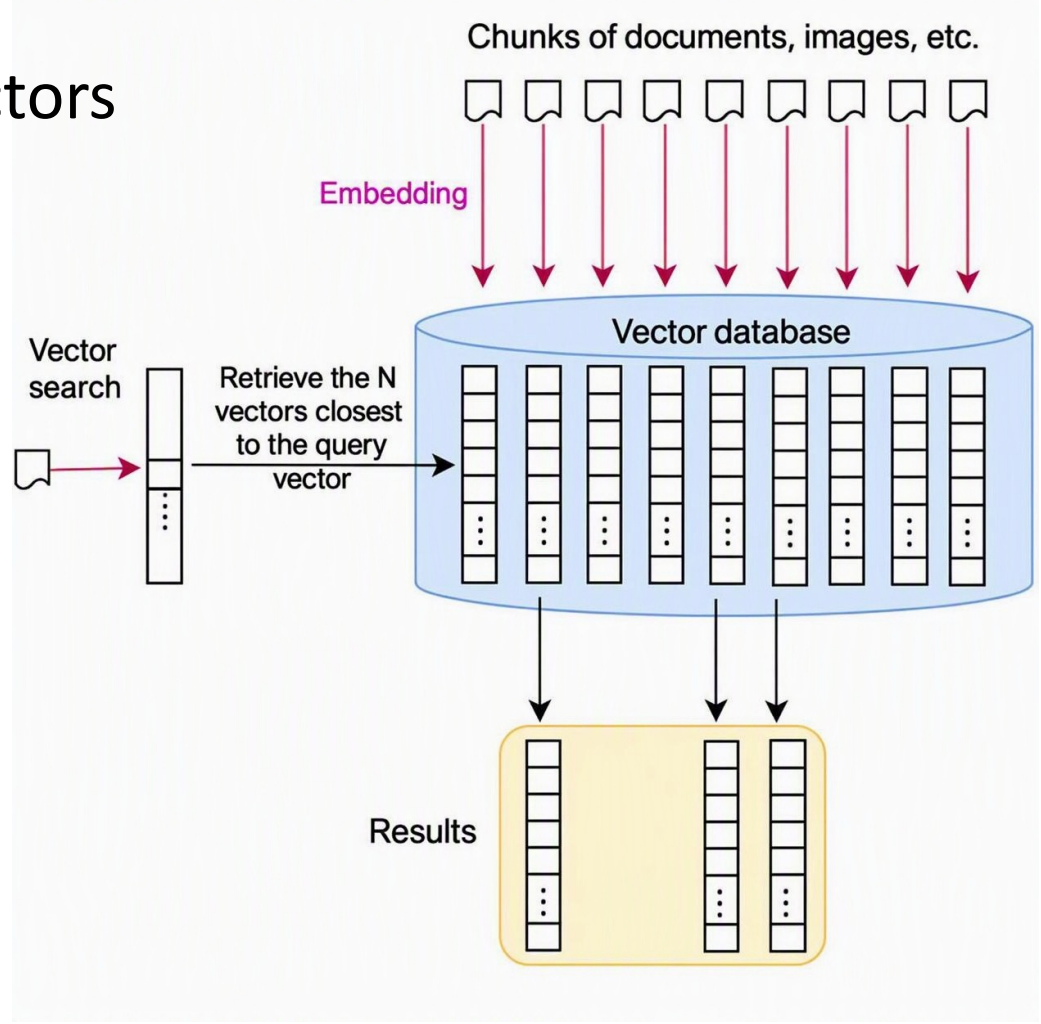
Session Code: F202



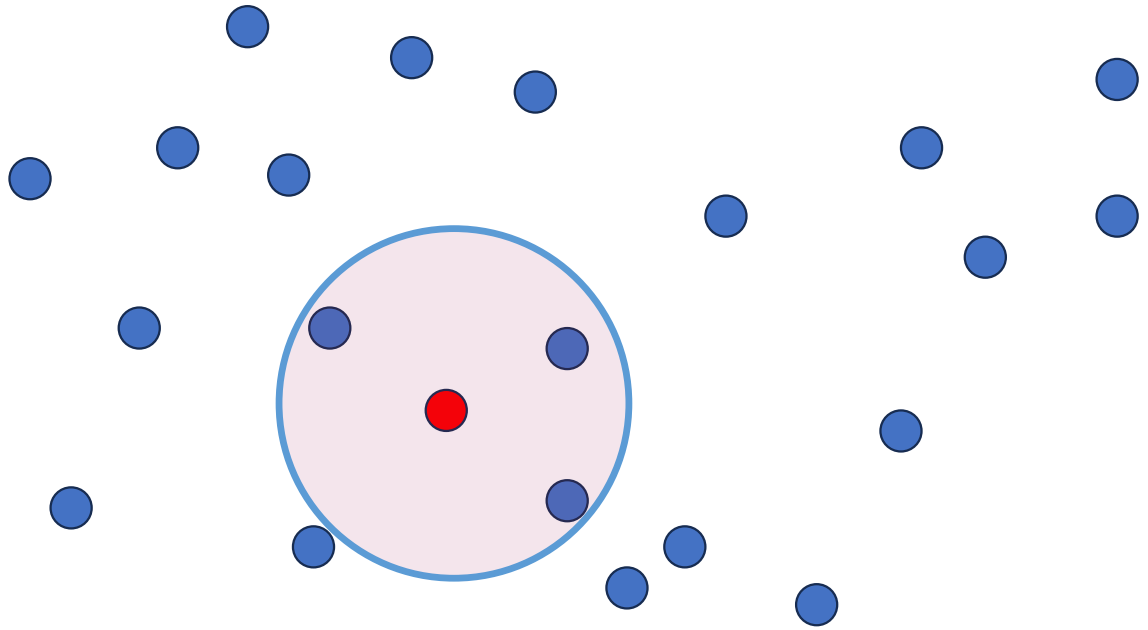
Abstract

This hands-on lab will explore how Db2 integrates vector data with structured relational data to support AI-driven use cases. Through guided exercises, attendees will combine unstructured text-derived vectors with traditional data and apply similarity search across both. The lab shows queries that merge SQL filtering with vector similarity to enable semantic search, recommendations, and intelligent data retrieval.

Simple queries with vectors



What are the N vectors closest to the query vector?



Example Vectors of Dimension n

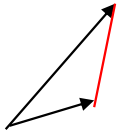
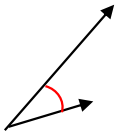
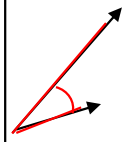
Vector U

u_0
u_1
u_2
u_3
u_4
u_5
u_6
$\vdots$
u_{n-1}

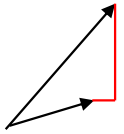
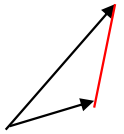
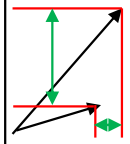
Vector V

v_0
v_1
v_2
v_3
v_4
v_5
v_6
$\vdots$
v_{n-1}

Common Distance/Similarity Metrics

Metric	Vectors u and v	Value range	Interpretation	Influencing variables	
Euclidean (L2)	$\sqrt{\sum_{i=0}^{n-1}(u_i - v_i)^2}$	$0, \ldots, \infty$	Higher values mean lower similarity	Euclidean distance	
Cosine	$\frac{\sum_0^{n-1}(u_i \cdot v_i)}{\sqrt{\sum_0^{n-1} u_i^2} \cdot \sqrt{\sum_0^{n-1} v_i^2}}$ Length	$-1, \ldots, 1$	Higher values mean greater similarity	Angle	
Dot product	$\sum_0^{n-1}(u_i \cdot v_i)$	$-\infty, \ldots, \infty$	Higher values mean greater similarity	Angles and vector lengths	

Other Common Distance/Similarity Metrics

Metric	Vectors u and v	Value range	Interpretation	Influencing variables	
Manhattan	$\sum_{i=0}^{n-1} \lvert u_i - v_i \rvert$	$0, \dots, \infty$	Higher values mean lower similarity	Euclidean distance	
Squared Euclidean	$\sum_{i=0}^{n-1} (u_i - v_i)^2$	$0, \dots, \infty$	Higher values mean lower similarity	Euclidean distance	
Hamming	$\left\lvert \left\{ \begin{matrix} (u_i, v_i): u_i \neq v_i \\ \text{for } i \in \{0, \dots, n-1\} \end{matrix} \right\} \right\rvert$	$0, \dots, \mathrm{n} - 1$	Higher values mean lower similarity	Number of different vector components	

Example: Vector Search and Hamming Distance (1)

- A company offers N different products.
- Customer k buys shopping cart W_k.
- The aim is to find customers with shopping behavior similar to customer k_0.

Possible solution:

- Represent the product set and its subsets, such as shopping carts W_k, as a vector v with N dimensions.
- If product i is in customer k's shopping cart W_k, set component v_i of vector v to 1. Otherwise, set it to 0.

Vector Search and Hamming Distance (2)

	w_{k_0}	w_{k_1}	w_{k_2}	w_{k_3}	w_{k_4}
Fruit	0	0	1	0	0
Vegetables	1	0	1	0	1
Meat	1	0	1	0	1
Fish	0	0	1	1	1
Sausage	1	0	1	0	1
Flour	0	0	0	1	0
Sugar	0	1	0	0	1
Salt	0	0	0	1	1

Vector Search and Hamming Distance (3)

	w_{k_0}		w_{k_1}
Fruit	0	$=$	0
Vegetables	1	$\neq$	0
Meat	1	$\neq$	0
Fish	0	$=$	0
Sausage	1	$\neq$	0
Flour	0	$=$	0
Sugar	0	$\neq$	1
Salt	0	$=$	0

Hamming distance: 4

Vector Search and Hamming Distance (4)

	w_{k_0}		w_{k_2}
Fruit	0	$\neq$	1
Vegetables	1	$=$	1
Meat	1	$=$	1
Fish	0	$\neq$	1
Sausage	1	$=$	1
Flour	0	$=$	0
Sugar	0	$=$	0
Salt	0	$=$	0

Hamming distance: 2

Vector Search and Hamming Distance (5)

	w_{k_0}		w_{k_3}
Fruit	0	=	0
Vegetables	1	≠	0
Meat	1	≠	0
Fish	0	≠	1
Sausage	1	≠	0
Flour	0	≠	1
Sugar	0	=	0
Salt	0	≠	1

Hamming distance: 6

Vector Search and Hamming Distance (6)

	w_{k_0}		w_{k_4}
Fruit	0	=	0
Vegetables	1	=	1
Meat	1	=	1
Fish	0	≠	1
Sausage	1	=	1
Flour	0	=	0
Sugar	0	≠	1
Salt	0	≠	1

Hamming distance: 3

Vector Search and Hamming Distance (7)

	w_{k_0}	w_{k_1}	w_{k_2}	w_{k_3}	w_{k_4}
Fruit	0	0	1	0	0
Vegetables	1	0	1	0	1
Meat	1	0	1	0	1
Fish	0	0	1	1	1
Sausage	1	0	1	0	1
Flour	0	0	0	1	0
Sugar	0	1	0	0	1
Salt	0	0	0	1	1
Hamming distance:		4	2	6	3

Most similar cart

Most Similar Vector

Can this also be solved with pure SQL? (1)

```
CREATE TABLE t (
  id          INTEGER,
  fruit       SMALLINT,
  vegetables  SMALLINT,
  meat        SMALLINT,
  fish        SMALLINT,
  sausage     SMALLINT,
  flour       SMALLINT,
  sugar SMALLINT,
  salt SMALLINT
);
```

- Mapping of the components of a vector to individual columns.
- Each column corresponds to a dimension

```
INSERT INTO t VALUES (1, 0,0,0,0,0,0,1,0);
INSERT INTO t VALUES (2, 1,1,1,1,1,0,0,0);
INSERT INTO t VALUES (3, 0,0,0,1,0,1,0,1);
INSERT INTO t VALUES (4, 0,1,1,1,1,0,1,1);
```

Can SQL find similar vectors? (2)

```
SELECT
id,
CASE WHEN fruit      <> 0 THEN 1 ELSE 0 END +
CASE WHEN vegetables <> 1 THEN 1 ELSE 0 END +
CASE WHEN meat       <> 1 THEN 1 ELSE 0 END +
CASE WHEN fish       <> 0 THEN 1 ELSE 0 END +
CASE WHEN sausage    <> 1 THEN 1 ELSE 0 END +
CASE WHEN flour      <> 0 THEN 1 ELSE 0 END +
CASE WHEN sugar      <> 0 THEN 1 ELSE 0 END +
CASE WHEN salt       <> 0 THEN 1 ELSE 0 END hamming_distance
FROM t
ORDER BY 2
FETCH FIRST 1 ROWS ONLY;
```

Query vector

Can SQL find similar vectors? (3)

Result:

```
ID           HAMMING_DISTANCE
------------ ----------------
2            2
```

Euclidean distance in SQL

```
SELECT
  id,
  SQRT(
    POWER(fruit      - 0, 2) +
    POWER(vegetables - 1, 2) +
    POWER(meat       - 1, 2) +
    POWER(fish       - 0, 2) +
    POWER(sausage    - 1, 2) +
    POWER(flour      - 0, 2) +
    POWER(sugar      - 0, 2) +
    POWER(salt       - 0, 2)) euclidean_distance
FROM t
ORDER BY 2
FETCH FIRST 1 ROWS ONLY;
```

Query vector

Problems with this approach

- For vectors with hundreds or thousands of components, the SQL statement becomes very large and needs code generation.
- High-dimensional vectors may exceed the database limit on the number of columns.
- Without index support, the query compares the query vector with every stored vector. Large datasets can therefore lead to long query runtimes.

Solving these problems

- Return approximate results for the k vectors closest to query vector s.
- Efficient index use becomes possible.
- How good is the approximation?
- Recall measures the quality of the results.

Definition of Recall

- Let V be the set of all vectors.
- Let Q be the set of k vectors returned by approximate search for query vector s.
- Let K be the set of k vectors truly closest to query vector s.
- Recall r is defined as follows:

$$\frac{|Q \cap K|}{k}$$

Similarity Search vs Exact Search



```
SELECT
  id,
  price
FROM vehicle
WHERE
  make            = 'Mercedes-Benz'  AND
  series          = 124 AND
  model           = 'E 200' AND
  body_shape      = 'sedan' AND
  fuel            = 'diesel' AND
  gearbox         = 'manual gearbox' AND
  color           = 'Light Blue'
;
```

- SQL query for exact result does not return any hits
- Similarity search also returns similar vehicles, e.g., model 'E 220' or series 123

AutoNova makes hybrid search tangible

LAB PROPOSAL / SEMANTIC USED-CAR MARKETPLACE / DB2 12.1.5

“A reliable family car that’s cheap to run.”

The request is meaningful—but not a database filter.

STRUCTURED FACTS

Price · mileage · year

Fuel type · accident history

Dealer location

Precise constraints → SQL filters

HUMAN LANGUAGE

Listing descriptions

Mechanic notes · owner reviews

Policy FAQs

Vague intent → vector similarity

SQL filters + vector similarity = cars that fit both

Requirements

ENVIRONMENT & DATA CONTRACT

01 **Platform —** Linux x86_64 (AMD64) only; no pureScale or DPF. Apple Silicon/arm64 cannot run Exercise 6.

02 **Table shape —** Use plain row-organized, non-partitioned tables.

03 **Vector column —** One per table; NOT NULL; no DEFAULT; never in a PK, unique key, FK, or other index. Use a surrogate integer key.

04 **Type + metric —** Use FLOAT32 and COSINE everywhere. The ORDER BY VECTOR_DISTANCE metric must match the index metric.

05 **Capacity —** A 384-dimension model needs an 8K table space: 4K → 256, 8K → 512, 16K → 1024 FLOAT32 dimensions.

06 **Data validation —** Require VECTOR_NORM(...) > 0 before a COSINE build to avoid SQL0801N.

BUILD, OPERATE & VERSION

07 **Build order —** Bulk load first; create the vector index only afterwards.

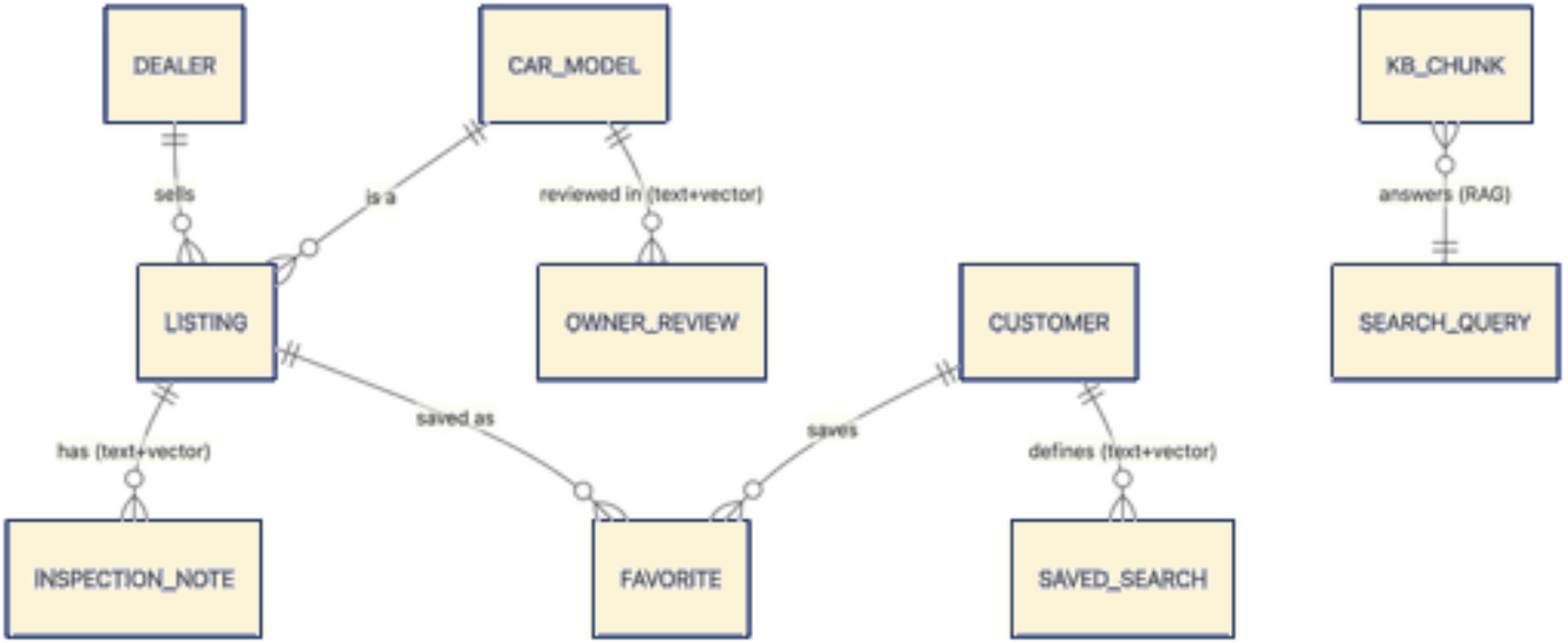
08 **After indexing —** Do not LOAD or REORG. TRUNCATE leaves the table unusable until the index is dropped.

09 **Offline build —** Creation takes a Z-lock. Index only the instructor-prepared 100k-row table while attendees read the next page.

10 **Deployment —** db2look does not capture vector indexes; keep CREATE VECTOR INDEX in its own script.

11 **Version split —** VECTOR and VECTOR_DISTANCE start in 12.1.2; TO_EMBEDDING and external models are documented as built-in. Exercises 0–5 and 7 run on 12.1.2+; Exercise 6 needs 12.1.5.

Data Model Used



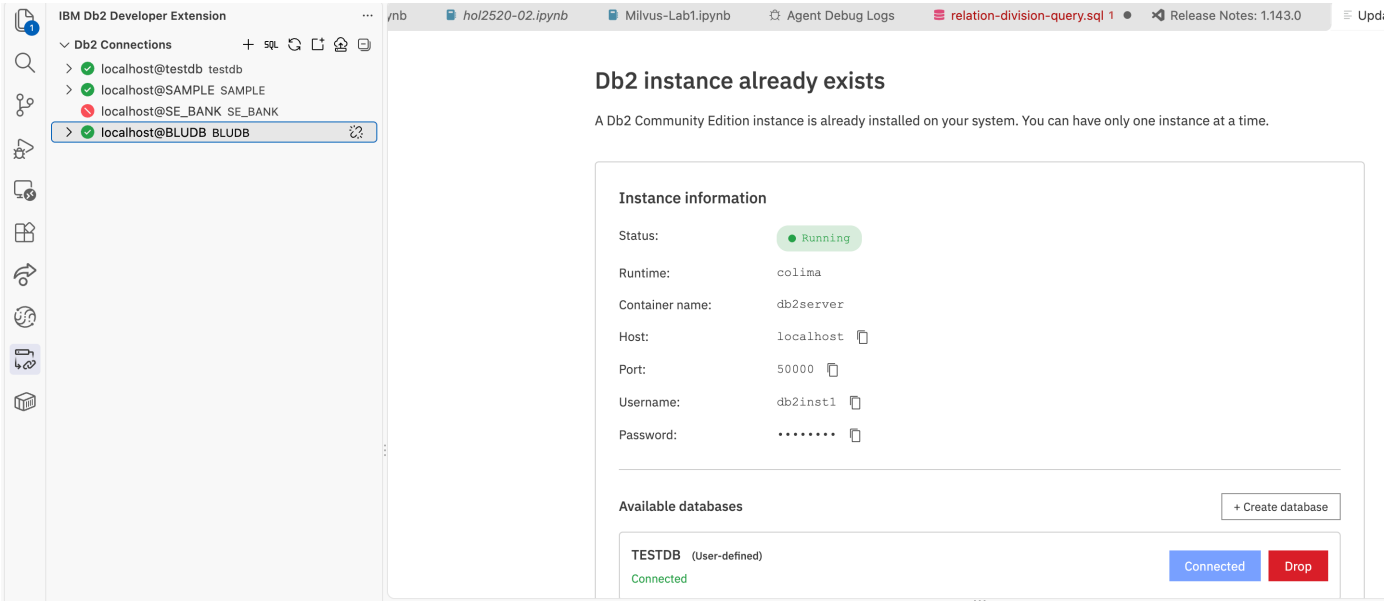
AutoNova schema inventory

Table	Structured columns	Text + vector	Rows
DEALER	dealer_id, name, city, state, latitude, longitude, rating	–	**40**
CAR_MODEL	model_id, make, model, body_type, seats, original_msrp	–	**~120**
LISTING	listing_id, vin, model_id, dealer_id, model_year, mileage, price, fuel_type, transmission, color, accident_free, owners, status, listed_date	description and desc_embedding VECTOR(384,FLOAT32) NOT NULL	**5,000 (scaled to 100k for Exercise 6)**
INSPECTION_NOTE	note_id, listing_id, inspected_on, category, severity (1–5)	note_text and note_embedding	**~15,000**
OWNER_REVIEW	review_id, model_id, model_year, rating, mileage_at_review	review_text and review_embedding	**~10,000**
CUSTOMER	customer_id, name, state, home_lat, home_lon, budget_max	–	**~500**
FAVORITE	customer_id, listing_id, fav_ts	–	**~5,000**
SAVED_SEARCH	saved_search_id, customer_id, max_price, max_mileage, min_year	search_text and search_embedding	**~300**
KB_CHUNK	chunk_id, doc_title, topic, applies_to_state	chunk_text and chunk_embedding	**~60**
SEARCH_QUERY	query_id, query_text	query_embedding	**~15**

Lab Preparation Steps

Get a local Db2 instance, e.g., via the Visual Studio Code Db2 Developer Extension

https://ibm.github.io/db2developerextension-about/



Get the Lab Files

Clone the github repository:

```
git clone https://github.com/andreasweininger/idugna2026-f202.git
```

Use Local Data Models

Installation of ollama on local machine (e.g., on MacOS):

```
> brew install ollama
```

Start ollama:

```
> ollama serve &
```

Get embedding and text generation models (others are also possible):

```
> ollama pull nomic-embed-text

> ollama pull llama3.2
```

Prepare Database

- Db2 12.1.5 instance on Linux x86_64 necessary
- Simple way to get such an instance on your notebook (even on MacOS with ARM processors) is via the Visual Studio Code Db2 Developer Extension: https://marketplace.visualstudio.com/items?itemName=IBM.db2-for-luw
- Create database with page sizes that are sufficiently large. Simple way to achieve this (also all of our tables will be row organized):

```
db2set DB2_WORKLOAD=ANALYTICS
```

TEXT_EMBEDDING Model

```
CREATE EXTERNAL MODEL IDUGNA26.local_embedder
PROVIDER OPENAI
KEY 'ollama'
ID 'nomic-embed-text'
TYPE TEXT_EMBEDDING RETURNING VECTOR(768, FLOAT32)
URL 'http://host.docker.internal:11434/v1/embeddings';
```

IDUGNA26-create-external-models.sql

TEXT_GENERATION Model

```
CREATE EXTERNAL MODEL IDUGNA26.local_llm
PROVIDER OPENAI
KEY 'ollam'
ID 'llama3.2'
TYPE TEXT_GENERATION RETURNING VARCHAR(5000)
URL 'http://host.docker.internal:11434/v1/chat/completions';
```

IDUGNA26-create-external-models.sql

Checking Models

```
SELECT modelschema,
       modelname,
       provider,
       modelid,
       modeltype,
       returntype
FROM syscat.externalmodels
WHERE modelschema = 'IDUGNA26';
```

MODELSCHEMA	MODELNAME	PROVIDER	MODELID	MODELTYPE	RETURNTYPE
IDUGNA26	LOCAL_EMBEDDER	OPENAI	nomic-embed-text	TEXT_EMBEDDING	VECTOR(768,FLOAT32)
IDUGNA26	LOCAL_LLM	OPENAI	llama3.2	TEXT_GENERATION	VARCHAR(5000 OCTETS)

IDUGNA26-create-external-models.sql

Create Tables

- Script: IDUGNA26-create-tables.sql
- All tables are row organized
- Tables created in such an order that is compatible with the referential constraints

Create Staging Tables

- Script: IDUGNA26-create-staging-tables.sql
- They are used for easier importing of precomputed vectors

Insert Reference Data and Create Views

- Insert reference data: IDUGNA26-insert-reference-data.sql
- Create views: IDUGNA26-create-views.sql

Importing Tables

- Import of tables should run on the database server itself.
- For the Db2 instance deployed Visual studio Code Db2 Developer Extension this means you have to enter the container.

```
docker exec -ti db2server bash -c "su - db2inst1"
```

- Load files from data directory must be made available in a directory /database/loadfiles or the import statement have to be adopted accordingly

```
db2 connect to databasename
db2 set schema idugna26
```

- Concatenate files listing_scale0.del and listing_scale1.del to listing_scale.del

Importing Tables and Copy from Staging Tables

- Run these import statements

```
db2 IMPORT FROM /database/loadfiles/dealer.del OF DEL INSERT INTO dealer
db2 IMPORT FROM /database/loadfiles/car_model.del OF DEL INSERT INTO car_model
db2 IMPORT FROM /database/loadfiles/customer.del OF DEL INSERT INTO customer
db2 IMPORT FROM /database/loadfiles/listing.del OF DEL INSERT INTO stg_listing
db2 IMPORT FROM /database/loadfiles/inspection_note.del OF DEL INSERT INTO stg_inspection_note
db2 IMPORT FROM /database/loadfiles/owner_review.del OF DEL INSERT INTO stg_owner_review
db2 IMPORT FROM /database/loadfiles/saved_search.del OF DEL INSERT INTO stg_saved_search
db2 IMPORT FROM /database/loadfiles/kb_chunk.del OF DEL INSERT INTO stg_kb_chunk
db2 IMPORT FROM /database/loadfiles/search_query.del OF DEL INSERT INTO stg_search_query
```

- Copy from staging tables to real tables: IDUGNA26-move-from-staging-tables.sql
- Run the remaining import statements:

```
db2 IMPORT FROM /database/loadfiles/favorite.del OF DEL INSERT INTO favorite
db2 IMPORT FROM /database/loadfiles/listing_incoming.del OF DEL INSERT INTO stg_listing_incoming
```

The preparation of the environment is finished now

Run the lab

- Instructions are in the file exercises.md
- Solutions are in the directory solutions

IDUG

2026

Atlanta | October 26 - 29

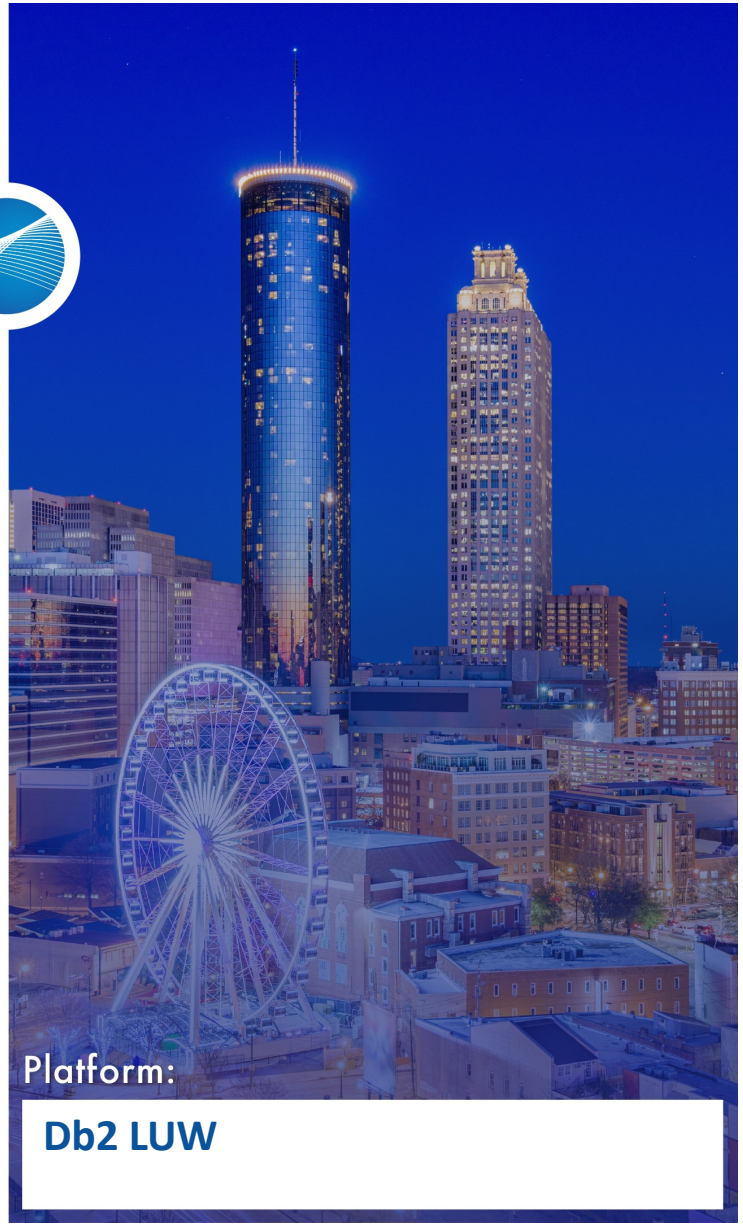
NA Db2 TECH CONFERENCE

Bridging Structured and Unstructured Data: Hybrid AI Queries with Db2

Andreas Weininger & Wilfried Hoge, *IBM*

Contact: {Andreas.Weininger|hoge}@de.ibm.com

Session Code: F202




IDUG
2026 NA Db2 Tech Conference